# REDES NEURONALES CONVOLUCIONALES

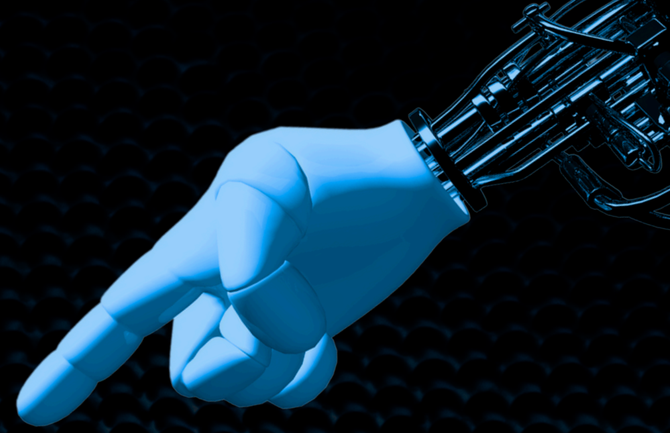
Michael Franco

&

Michael Lata

# Índice


1. Introducción breve a redes neuronales
2. Descripcion de Dataset
3. Análisis exploratorio/ visualización de imágenes
4. Entrenamiento y optimización de una red neuronal
5. Evaluación del modelo presentando medidas de calidad
6. Función de predicción de una nueva imagen
7. Conclusiones
8. Referencias



# Introducción

- La clasificación automática de imágenes es un reto fundamental en inteligencia artificial y visión por computadora.
- Las redes neuronales convolucionales (CNN) han revolucionado el procesamiento de imágenes, permitiendo extraer características complejas y realizar tareas de reconocimiento con alta precisión.
- En este proyecto, se aborda la clasificación de imágenes de alimentos utilizando el dataset Food-101, aplicando técnicas modernas de deep learning.

# Descripcion Dataset

## Dataset

-Problema: Clasificar imágenes de alimentos en 101 categorías distintas, enfrentando el reto de la alta similitud visual entre algunos platillos.

- Dataset Food-101:
  - 101,000 imágenes a color, organizadas en 101 clases (platillos).
  - División: 75,750 imágenes para entrenamiento (750 por clase), 25,250 para prueba (250 por clase).
  - Imágenes variadas en tamaño y presentación, etiquetas en formato string (nombre del platillo).
  - Ejemplos de clases: pizza, pad_thai, guacamole, chicken_curry, ramen, etc.
- Importancia: El reconocimiento automático de alimentos tiene aplicaciones en nutrición, salud, restauración y análisis de hábitos alimenticios.

# Analisis Exploratorio y Preparación del dataset

Se realizó una exploración visual del dataset para comprender la diversidad y calidad de las imágenes.

Visualización de imágenes aleatorias y específicas, mostrando sus dimensiones, valores de píxeles y etiquetas asociadas.

Se identificaron variaciones en iluminación, ángulos y estilos de presentación, lo que representa un desafío adicional para la clasificación.

Este análisis permitió detectar posibles fuentes de confusión y la necesidad de técnicas robustas de preprocesamiento y modelado.

-Redimensionamiento: Todas las imágenes se ajustaron a un tamaño estándar (por ejemplo, 100x100 o 224x224 píxeles) para ser compatibles con la arquitectura de la CNN.

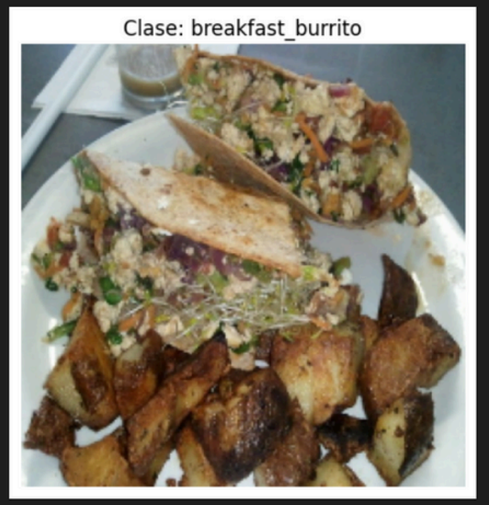
Normalización: Los valores de los píxeles se escalaron al rango [0, 1] para mejorar la estabilidad y velocidad del entrenamiento.

Codificación de etiquetas: Las etiquetas de texto se transformaron a formato numérico para su uso en el modelo.

Aumento de datos: Se aplicaron técnicas como rotación, traslación y volteo para incrementar la diversidad del conjunto de entrenamiento y mejorar la generalización.

División de datos: El dataset se separó en conjuntos de entrenamiento y validación, asegurando una evaluación objetiva del modelo.

```
Found 101000 files belonging to 101 classes.
Imagen 9999
Forma (shape): (224, 224, 3)
Valor mínimo: 0.0, máximo: 255.0
Etiqueta ID: 9 - Clase: breakfast_burrito
```




```
Found 101000 files belonging to 101 classes.
Imagen 10
Forma (shape): (100, 100, 3)
Valor mínimo: 0.0, máximo: 255.0
Etiqueta ID: 0 - Clase: apple_pie
```

# Entrenamiento y optimización de la red neuronal

- Arquitectura: Se diseñó una CNN profunda con varias capas convolucionales, de pooling, normalización por lotes y dropout para prevenir el sobreajuste.

- Compilación: Se utilizó la función de pérdida `sparse_categorical_crossentropy` y el optimizador Adam.

- Entrenamiento: El modelo se entrenó durante varias épocas, utilizando callbacks como EarlyStopping y ModelCheckpoint para evitar el sobreentrenamiento y guardar el mejor modelo.

- Fine-tuning: Se aplicó ajuste fino descongelando capas superiores de un modelo preentrenado, permitiendo adaptar mejor las representaciones a las características del dataset Food-101.

- Resultados intermedios: Se monitoreó la evolución de la pérdida y la precisión en entrenamiento y validación.

```python
# Entrenamiento
history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=15,
    callbacks=[early_stop, checkpoint, csv_logger]
)
```

```
Found 101000 files belonging to 101 classes.
Using 80800 files for training.
Found 101000 files belonging to 101 classes.
Using 20200 files for validation.
Clases (ejemplo): ['apple_pie', 'baby_back_ribs', 'baklava', 'beef_carpaccio', 'beef_tartare', 'beet_salad', 'beignets', 'bibimbap', 'bread_pudding', 'breakfast_burrito']
Número total clases: 101
```

```
Epoch 1/15
1263/1263 [==============================] - 100s 39ms/step - loss: 4.6498 - accuracy: 0.1802 - val_loss: 3.4343 - val_accuracy: 0.3476
Epoch 2/15
1263/1263 [==============================] - 125s 63ms/step - loss: 3.6346 - accuracy: 0.3139 - val_loss: 3.1490 - val_accuracy: 0.3989
Epoch 3/15
1263/1263 [==============================] - 61s 36ms/step - loss: 3.3077 - accuracy: 0.3631 - val_loss: 2.9977 - val_accuracy: 0.4208
Epoch 4/15
1263/1263 [==============================] - 57s 34ms/step - loss: 3.0796 - accuracy: 0.3959 - val_loss: 2.8945 - val_accuracy: 0.4351
Epoch 5/15
1263/1263 [==============================] - 57s 33ms/step - loss: 2.9123 - accuracy: 0.4194 - val_loss: 2.8155 - val_accuracy: 0.4428
Epoch 6/15
1263/1263 [==============================] - 57s 33ms/step - loss: 2.7778 - accuracy: 0.4391 - val_loss: 2.7531 - val_accuracy: 0.4449
Epoch 7/15
1263/1263 [==============================] - 56s 32ms/step - loss: 2.6678 - accuracy: 0.4539 - val_loss: 2.6986 - val_accuracy: 0.4508
Epoch 8/15
1263/1263 [==============================] - 56s 33ms/step - loss: 2.5623 - accuracy: 0.4682 - val_loss: 2.6608 - val_accuracy: 0.4526
```

# Evaluación del modelo y medidas de calidad

- Métricas calculadas: Accuracy, precision, recall y F1-score, tanto globales como por clase.

- Matriz de confusión: Visualización detallada de los aciertos y errores para cada clase, identificando patrones de confusión.

- Reporte de clasificación: Análisis de desempeño individual por clase, detectando categorías con mejor y peor rendimiento.

- Interpretación: El modelo logra buenos resultados generales, pero presenta dificultades en clases visualmente similares o con menor representación.

| | real | predicción | acierto |
|---|---|---|---|
| 2 | pho | pho | True |
| 3 | edamame | edamame | True |
| 4 | tuna_tartare | tuna_tartare | True |
| 6 | beef_carpaccio | beef_carpaccio | True |
| 7 | sashimi | sashimi | True |
| 17 | beignets | beignets | True |
| 18 | beignets | beignets | True |
| 19 | greek_salad | greek_salad | True |
| 20 | pho | pho | True |
| 22 | pancakes | pancakes | True |

```
Resultados de evaluación:
- Loss: 2.5069
- Accuracy: 0.4550
```

| | precision | recall | f1-score | support |
|---|---|---|---|---|
| apple_pie | 0.27 | 0.18 | 0.21 | 205 |
| baby_back_ribs | 0.44 | 0.45 | 0.44 | 215 |
| baklava | 0.46 | 0.45 | 0.45 | 213 |
| beef_carpaccio | 0.51 | 0.57 | 0.54 | 207 |
| beef_tartare | 0.26 | 0.26 | 0.26 | 190 |
| beet_salad | 0.33 | 0.36 | 0.34 | 203 |
| beignets | 0.53 | 0.70 | 0.60 | 191 |
| bibimbap | 0.67 | 0.66 | 0.67 | 209 |
| bread_pudding | 0.23 | 0.17 | 0.20 | 198 |
| breakfast_burrito | 0.38 | 0.33 | 0.35 | 216 |
| bruschetta | 0.37 | 0.38 | 0.38 | 193 |
| caesar_salad | 0.46 | 0.42 | 0.44 | 216 |
| cannoli | 0.41 | 0.40 | 0.41 | 212 |
| caprese_salad | 0.38 | 0.37 | 0.37 | 196 |
| carrot_cake | 0.46 | 0.40 | 0.42 | 189 |
| ceviche | 0.26 | 0.23 | 0.24 | 208 |
| cheese_plate | 0.36 | 0.42 | 0.39 | 195 |
| cheesecake | 0.33 | 0.31 | 0.32 | 189 |
| chicken_curry | 0.35 | 0.32 | 0.34 | 200 |
| chicken_quesadilla | 0.38 | 0.43 | 0.40 | 205 |
| chicken_wings | 0.57 | 0.51 | 0.54 | 199 |
| chocolate_cake | 0.46 | 0.45 | 0.45 | 212 |
| chocolate_mousse | 0.33 | 0.24 | 0.28 | 221 |
| ... | | | | |
| accuracy | | | 0.45 | 20200 |
| macro avg | 0.45 | 0.46 | 0.45 | 20200 |
| weighted avg | 0.45 | 0.45 | 0.45 | 20200 |

# Resultados obtenidos y análisis

```
Accuracy : 0.4550
Precision: 0.4521
Recall   : 0.4550
F1 Score : 0.4512
```

En la tabla presentada se comparan las clases reales y las predichas por el modelo para algunas imágenes del conjunto de validación. Se observan tanto aciertos como errores en la clasificación:

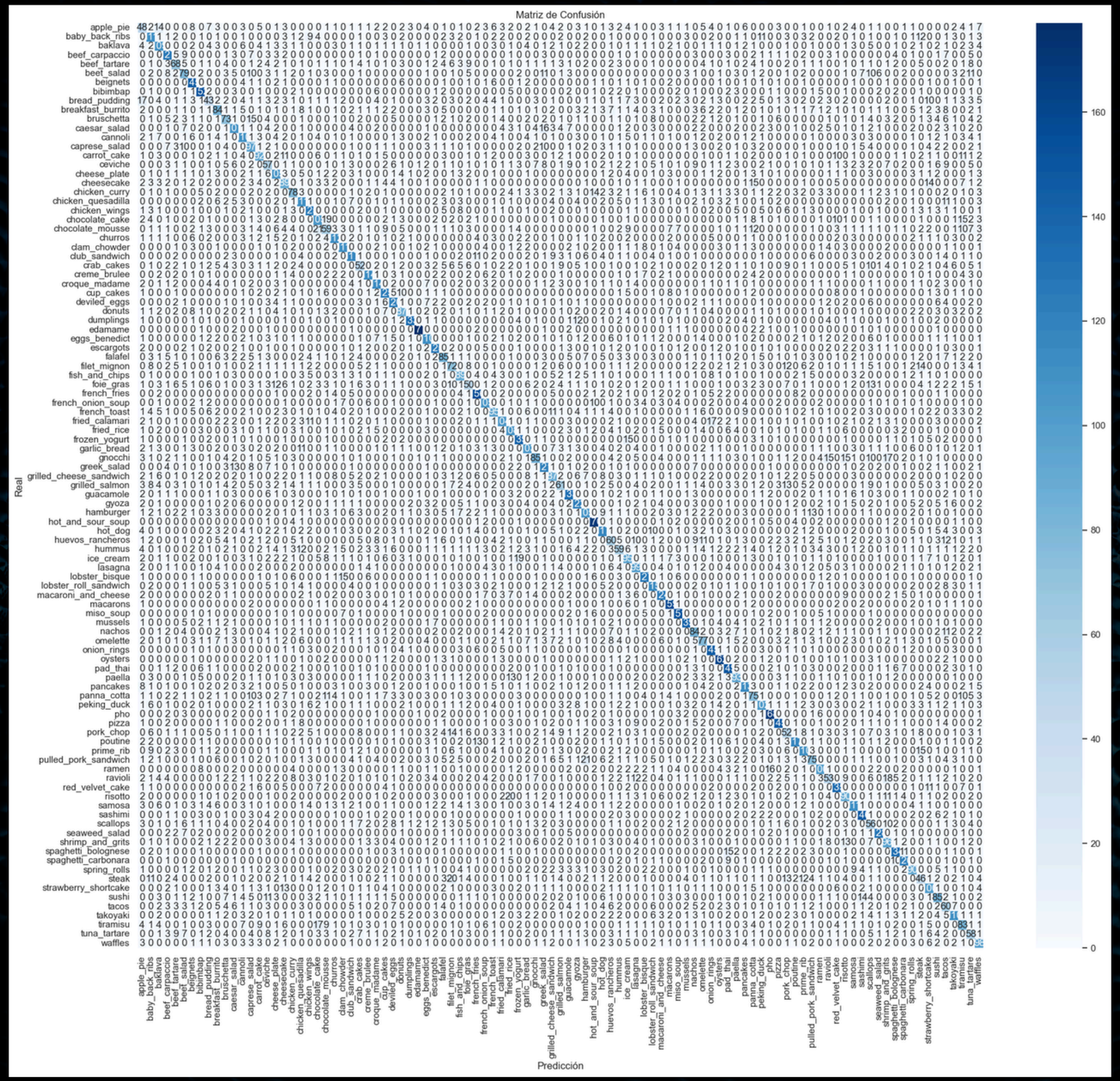
- Cuando la columna "real" coincide con "predicción" (por ejemplo, `oysters` - `oysters` o `strawberry_shortcake` - `strawberry_shortcake`), el modelo ha clasificado correctamente la imagen.

- En otros casos, el modelo confunde clases visualmente similares o relacionadas, como `steak` predicho como `filet_mignon`, o `cannoli` como `breakfast_burrito`. Estas confusiones pueden deberse a varias razones:

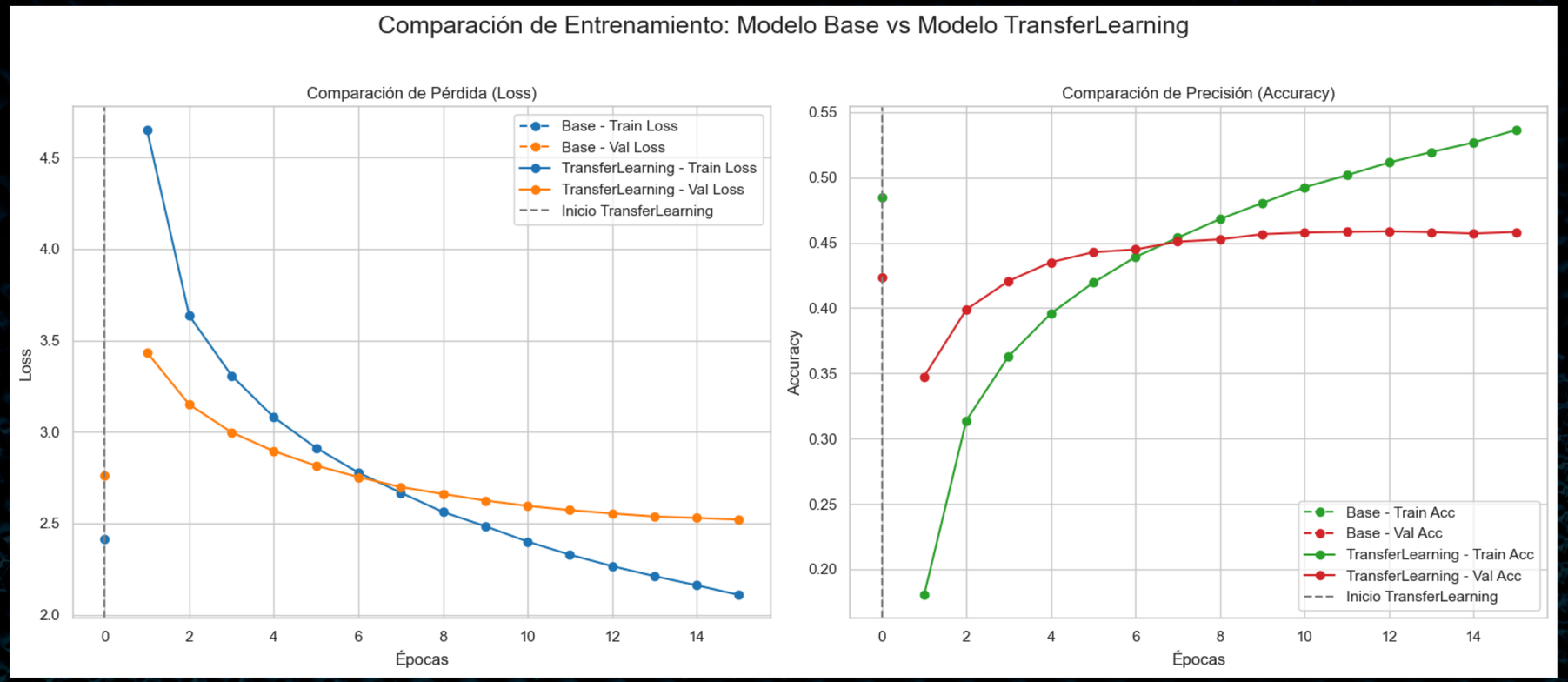
  - Similitud visual: Algunos platillos pueden compartir características visuales (color, forma, presentación), lo que dificulta su distinción incluso para humanos.
  - Variabilidad en las imágenes:Diferentes estilos de preparación, ángulos de toma o iluminación pueden hacer que una clase se parezca a otra.

  - Cantidad y diversidad de datos: Si algunas clases tienen menos ejemplos o mayor variabilidad interna, el modelo puede tener más dificultades para aprender a diferenciarlas.

  - Capacidad del modelo: Aunque la arquitectura utilizada es robusta, siempre existe un margen de error, especialmente en problemas multiclase con muchas categorías similares.



# Comparativa de resultados de  entrenamiento




Los gráficos muestran la comparación entre el entrenamiento del modelo base y el proceso de TransferLearning. En la gráfica de pérdida (Loss) se observa una disminución continua de la pérdida de entrenamiento (Train Loss) durante ambas etapas, lo que indica un aprendizaje progresivo. La pérdida de validación (Val Loss) se mantiene estable, pero mejora ligeramente después de iniciar el TransferLearning, lo que sugiere una mejor capacidad de generalización. La línea vertical en el gráfico indica el punto donde comienza el TransferLearning.

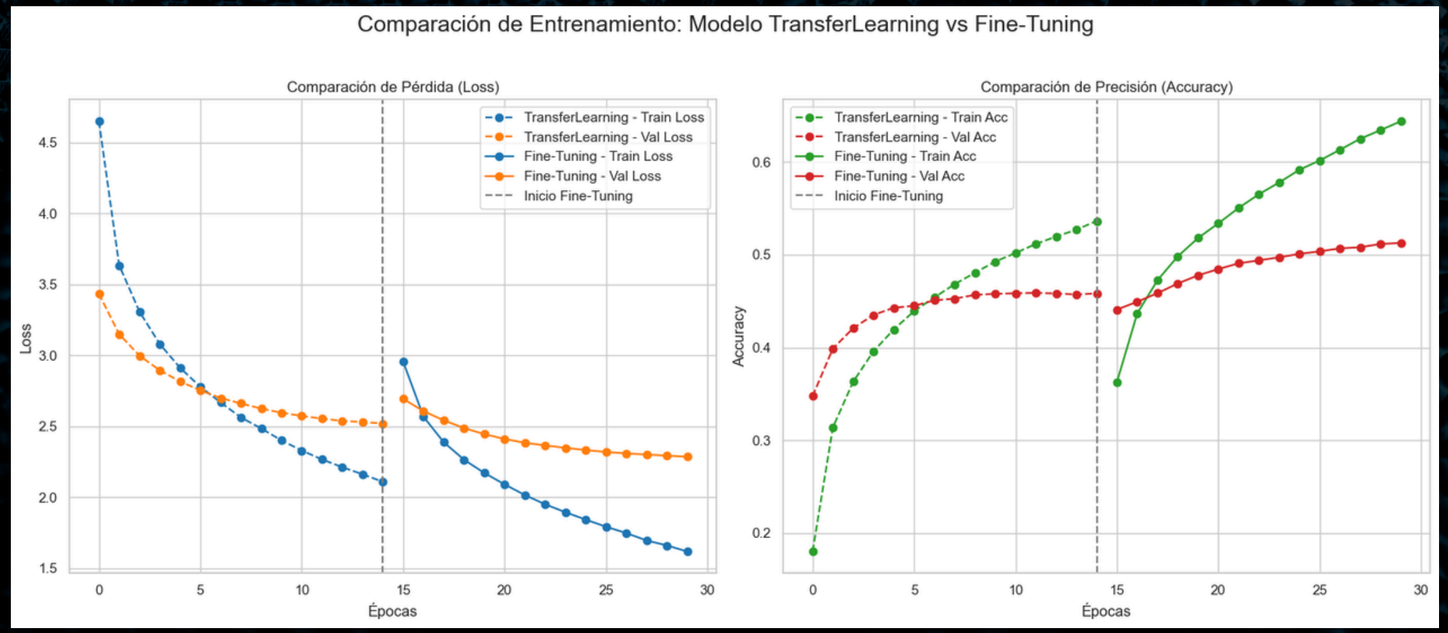
En la gráfica de precisión (Accuracy), la precisión de entrenamiento (Train Acc) mejora constantemente y alcanza valores más altos tras el  TransferLearning. La precisión de validación (Val Acc) también mejora, aunque de forma más moderada. En conjunto, los resultados muestran que el  TransferLearning tuvo un impacto positivo, mejorando el desempeño del modelo especialmente en entrenamiento, y generando una leve pero favorable mejora en validación.

# TransferLearning

| | real | predicción | acierto |
|---|---|---|---|
| 3 | frozen_yogurt | frozen_yogurt | True |
| 5 | hot_and_sour_soup | hot_and_sour_soup | True |
| 10 | huevos_rancheros | huevos_rancheros | True |
| 12 | seaweed_salad | seaweed_salad | True |
| 19 | pho | pho | True |
| 24 | steak | steak | True |
| 27 | ceviche | ceviche | True |
| 28 | spaghetti_carbonara | spaghetti_carbonara | True |
| 30 | onion_rings | onion_rings | True |
| 33 | beignets | beignets | True |

La tabla muestra ejemplos donde se comparan las clases reales y las predichas por el modelo. En varios casos, el modelo acierta; en otros, se confunde con clases visualmente similares, como steak y filet_mignon.

Estos errores pueden deberse a similitudes visuales, variaciones en las imágenes o una distribución desigual de datos. El modelo utiliza transfer learning, lo que significa que parte de una red preentrenada en millones de imágenes y se adapta a una nueva tarea con un conjunto más pequeño, mejorando el rendimiento sin necesidad de entrenar desde cero.




Comparación: Transfer Learning vs. Fine-Tuning

🔹 Transfer Learning
Usa pesos preentrenados como punto de partida.
Permite un entrenamiento rápido con buenos resultados iniciales.

🔸 Fine-Tuning
Ajusta las capas preentrenadas para una mayor precisión.
Se adapta mejor a las características específicas del dataset.
Si las métricas mejoran, indica una mayor especialización del modelo.

# Función de predicción de una nueva imagen

- Se implementó una función que permite cargar una imagen externa, preprocesarla y obtener la predicción del modelo.

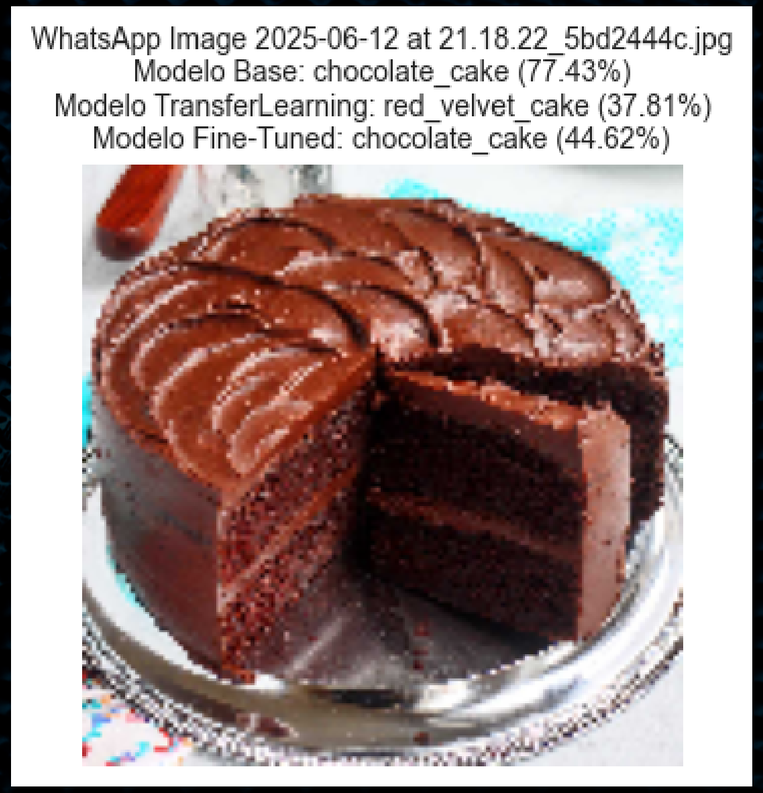
- Esta función facilita la aplicación práctica del modelo en situaciones reales, permitiendo clasificar imágenes fuera del conjunto original.

- Ejemplo de uso: Se muestra la imagen cargada y la clase predicha, validando la utilidad del sistema desarrollado.

Carga de imagen → Preprocesamiento → Predicción → Visualización de resultado.

Aplicación práctica: Clasificación real de alimentos capturados con cámara o buscados en línea.

Ejemplo: mostrar imagen cargada y etiqueta predicha con probabilidad.




La imagen muestra la predicción del modelo sobre una fotografía de un pastel de chocolate, clasificada como "chocolate_cake" con una confianza del 46.77%. Aunque la predicción es correcta, la baja confianza indica que el modelo no está completamente seguro, probablemente porque detecta características similares a otras clases.

Este resultado es útil para evaluar la capacidad del modelo de generalizar en imágenes nuevas. También señala posibles áreas de mejora, como ampliar el conjunto de datos de entrenamiento o ajustar la arquitectura del modelo para reforzar su precisión en casos visualmente similares.

# Mas predicción de varias imagenes que no estan en el dataset

WhatsApp Image 2025-06-12 at 21.18.20_3ac962c7.jpg
Modelo Base: pizza (30.11%)
Modelo TransferLearning: pizza (96.35%)
Modelo Fine-Tuned: pizza (99.91%)

WhatsApp Image 2025-06-12 at 21.18.19_51050236.jpg
Modelo Base: bruschetta (84.33%)
Modelo TransferLearning: bruschetta (53.21%)
Modelo Fine-Tuned: bruschetta (47.58%)

carbonara.jpg
Modelo Base: pad_thai (79.97%)
Modelo TransferLearning: spaghetti_carbonara (53.78%)
Modelo Fine-Tuned: spaghetti_carbonara (53.69%)

steaks.jpg
Modelo Base: chocolate_cake (56.52%)
Modelo TransferLearning: steak (58.07%)
Modelo Fine-Tuned: steak (50.09%)

seaweeb salad.jpg
Modelo Base: seaweed_salad (100.00%)
Modelo TransferLearning: seaweed_salad (97.91%)
Modelo Fine-Tuned: seaweed_salad (99.88%)

huevos rancheros.jpg
Modelo Base: huevos_rancheros (32.54%)
Modelo TransferLearning: huevos_rancheros (40.59%)
Modelo Fine-Tuned: huevos_rancheros (43.32%)

La función de predicción genera, para cada imagen, una clase estimada y un nivel de confianza. Estos resultados permiten interpretar qué tan seguro está el modelo al clasificar y detectar posibles errores o ambigüedades. Además, ayudan a evaluar el rendimiento real del modelo frente a datos nuevos y a identificar oportunidades de mejora.

# Resultados Comparativos

| Método | Accuracy | Loss |
|---|---|---|
| Modelo Base | 3.224 | 31.611 |
| Modelo Transfer Learning | 4.588 | 25.541 |
| Modelo Fine-Tuned | 5.126 | 22.849 |

El modelo base, entrenado desde cero, obtuvo el rendimiento más bajo.

El modelo con Transfer Learning mejoró notablemente la precisión inicial.

El modelo Fine-Tuned alcanzó el mejor desempeño, demostrando que ajustar los pesos del modelo preentrenado al nuevo dominio es altamente beneficioso.

## Interpretación de Resultados

**Modelo Base**

Entrenado completamente desde cero, sin conocimiento previo.
Presenta el peor desempeño en términos de precisión y pérdida.
Necesita más datos y tiempo para aprender patrones relevantes.

**Modelo con Transfer Learning**

Utiliza la arquitectura MobileNetV2 preentrenada con pesos de ImageNet.
Logra una mejora notable en precisión frente al modelo base.
La reducción en la pérdida indica un mejor ajuste al problema de clasificación.

**Modelo Fine-Tuned**

Parte del modelo con Transfer Learning, pero se descongelan capas para continuar el entrenamiento.
Alcanza el mejor desempeño general: mayor precisión y menor pérdida.
Refleja que el modelo ha aprendido características más específicas del nuevo dominio de datos.

# Resumen del Proyecto

Este proyecto abordó la clasificación multiclase de imágenes de alimentos mediante redes neuronales convolucionales (CNN), utilizando el dataset Food-101. Se desarrollaron todas las etapas del flujo de trabajo en visión por computadora: análisis exploratorio, preparación de datos, diseño y entrenamiento de modelos, evaluación y análisis de resultados.

***Principales Hallazgos***

**Eficacia de las CNN**
Las redes convolucionales fueron efectivas incluso con 101 clases y alta similitud visual entre categorías.

**Análisis exploratorio**
Permitió comprender la distribución de clases, visualizar ejemplos y anticipar problemas como el desbalance o la ambigüedad visual.

**Preparación de datos**
Técnicas como redimensionamiento, normalización y aumento de datos ayudaron a mejorar la generalización.

**Transfer Learning y Fine-Tuning**
El uso de modelos preentrenados (MobileNetV2) y su ajuste fino mejoró significativamente el rendimiento en comparación con un modelo entrenado desde cero.

**Evaluación y Aplicación**

**Curvas de entrenamiento**
Evidenciaron mejoras claras tras el fine-tuning, especialmente en la precisión de validación.

**Matriz de confusión**
Facilitó la identificación de clases con mayor confusión, principalmente por similitudes visuales.

**Predicción sobre imágenes externas**
El modelo generalizó adecuadamente, aunque con menor confianza en algunos casos.

**Predicción en tiempo real**
Se integró una función de inferencia en vivo con cámara web, demostrando su aplicabilidad práctica.

# Conclusiones

El proyecto demostró cómo las técnicas modernas de aprendizaje profundo, especialmente el uso de redes neuronales convolucionales, pueden abordar con éxito el desafío de la clasificación multiclase de imágenes en el dominio de alimentos. La implementación de Transfer Learning y Fine-Tuning permitió aprovechar el conocimiento previo de modelos preentrenados, logrando una mejora significativa en la precisión y la capacidad de generalización frente a modelos entrenados desde cero. Este enfoque no solo optimizó el proceso de entrenamiento, sino que también facilitó la adaptación del modelo a las particularidades del dataset Food-101, evidenciando la importancia de ajustar los pesos a contextos específicos para obtener mejores resultados.

Además, la integración de herramientas de visualización, pruebas con imágenes externas y la predicción en tiempo real mediante cámara web enriqueció el análisis y la validación del modelo. Estas estrategias permitieron identificar patrones de error, evaluar la robustez del sistema en condiciones reales y detectar posibles limitaciones, como la confusión entre clases visualmente similares o la sensibilidad a variaciones en las imágenes de entrada. En conjunto, el trabajo realizado pone de manifiesto tanto el potencial transformador de los modelos de deep learning en aplicaciones prácticas como los retos que aún persisten, subrayando la necesidad de un enfoque integral que combine técnicas avanzadas, análisis crítico y validación exhaustiva en escenarios reales.

# Referencias


- Bossard, L., Guillaumin, M., & Van Gool, L. (2014). Food-101 – Mining Discriminative Components with Random Forests. In European Conference on Computer Vision (ECCV). https://www.vision.ee.ethz.ch/datasets_extra/food-101/
- TensorFlow documentation: https://www.tensorflow.org/api_docs
- TensorFlow Datasets: https://www.tensorflow.org/datasets/catalog/food101
- Keras documentation: https://keras.io/
- NumPy documentation: https://numpy.org/doc/
- Matplotlib documentation: https://matplotlib.org/stable/contents.html
- scikit-learn documentation: https://scikit-learn.org/stable/documentation.html
- Seaborn documentation: https://seaborn.pydata.org/